

AnchorIQ — 해운 공급망 리스크 탐지 자동화 플랫폼

1. 프로젝트 소개

AnchorIQ는 11개 무료 API에서 선박/해운 데이터를 수집하고, Neo4j 온톨로지 그래프로 융합하여 제재국 연관 선박, 초크포인트 리스크, 항만 혼잡도를 탐지하는 플랫폼입니다. AI(OpenClaw → OpenAI Codex)가 리스크를 0~100점으로 판단하고, Kafka 이벤트 파이프라인과 n8n 워크플로우를 통해 Discord/Slack/Email로 자동 알림을 실행합니다.

핵심 기능

- 11개 외부 API(AIS/날씨/제재/뉴스/유가/환율/지정학/항만) 자동 수집 + Kafka 이벤트 스트림
- Neo4j 온톨로지 그래프 — 선박-회사-국가-항로-초크포인트 4홉 관계 탐색
- AI 리스크 스코어링 (0~100) + 자연어 질의 → Cypher 변환
- Kafka → n8n → Discord/Slack/Email 자동 알림 파이프라인
- 팔란티어 감성 대시보드 — 지도/그래프/위룸/타임라인 5개 뷰

담당 역할

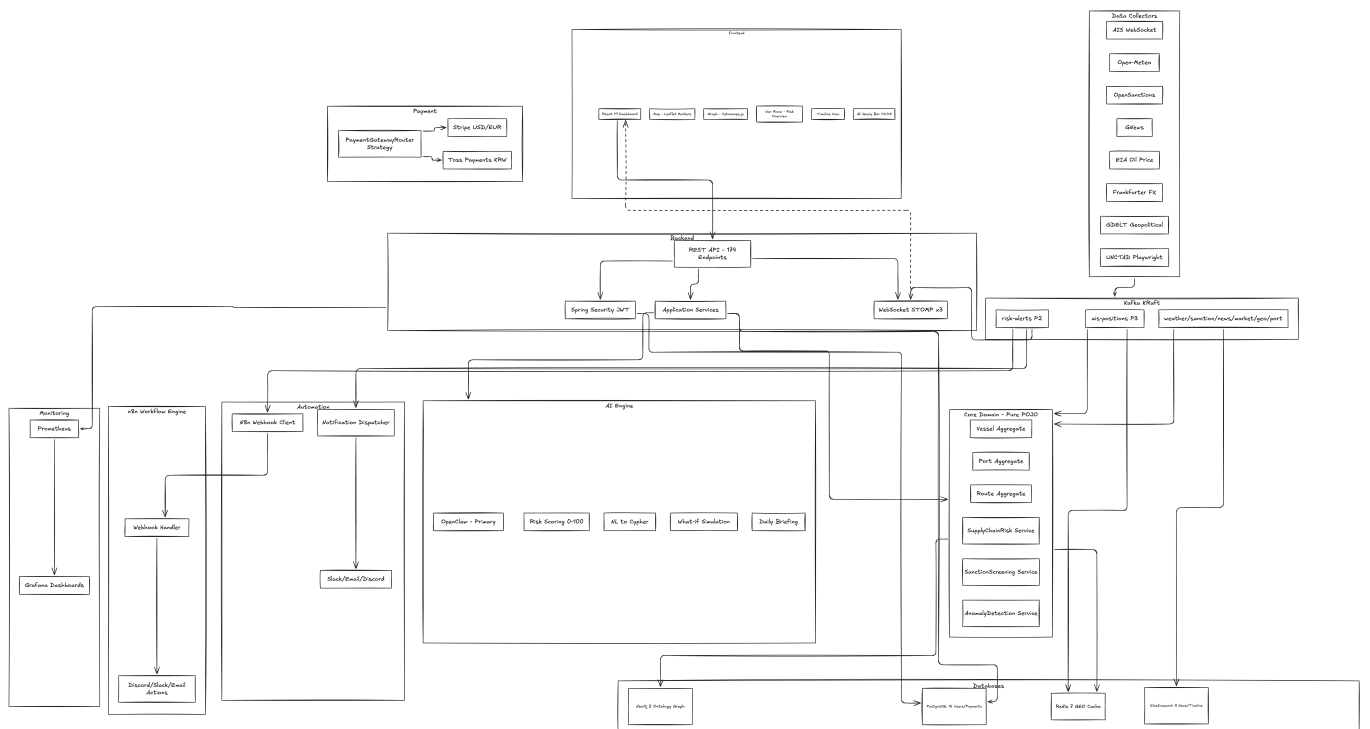
- 전체 아키텍처 설계 및 구현 (백엔드 + 프론트엔드 + 인프라)
- 21개 설계 문서 작성 후 AI 기반 Vibe Coding으로 구현
- DDD(풍부한 도메인 모델) + 헥사고널 아키텍처 적용
- Kafka 이벤트 파이프라인 + n8n 자동화 연동
- Claude Code + MCP(Playwright, Chrome DevTools, Figma) 활용 개발 워크플로

기술 스택

레이어	기술
Backend	Java 21, Spring Boot 3.4, Spring Security, Spring Data JPA/Neo4j/Redis, Spring Kafka
Frontend	React 19, TypeScript, Vite, Tailwind CSS, Leaflet, Cytoscape.js, Zustand
Database	Neo4j 5 (온톨로지), PostgreSQL 16 (유저/결제), Redis 7 (GEO/캐시), Elasticsearch 8 (검색)

Messaging	Kafka (KRaft 모드, 8개 토픽, 9개 Consumer)
AI	OpenClaw 게이트웨이 → OpenAI Codex 엔진
Automation	n8n (Webhook → Discord/Slack/Email)
Payment	Stripe + Toss Payments (Strategy 패턴)
Monitoring	Prometheus + Grafana
Infra	Docker Compose, Gradle 멀티 모듈 (5개 모듈)

시스템 아키텍처



2. 개발 방식 — 설계 주도 Vibe Coding

1인 풀스택으로 174개 API, 11개 데이터 수집기, 5개 대시보드 뷰를 구현해야 했습니다. 21개 설계 문서를 먼저 작성하고, Claude Code에 AGENTS.md(코딩 규칙 15개)와 CLAUDE.md(프로젝트 컨텍스트)를 통해 일관된 코드 생성을 강제했습니다.

설계는 직접, 구현은 AI에게

모든 기능을 구현 전에 마크다운 문서(.md)로 상세히 설계했습니다. API 엔드포인트 174개, 패키지 구조, DB 스키마, Kafka 토픽, 트랜잭션 전략까지 11개 Phase별 설계서(2,240줄)를 작성했고, AGENTS.md에 Clean Code + OOP + SOLID + DDD 규칙을 정의하여 AI가 항상 동일한 아키텍처와 컨벤션으로 코드를 생성하도록 제약을 걸었습니다.

"만들어줘"가 아니라 "이 설계서대로, 이 원칙대로 만들어라"라고 지시하는 방식이었고, 설계 문서가 충분히 상세했기 때문에 전체에 일관된 레이어 구조와 패턴이 유지되었습니다.

AI 도구 + MCP 활용

도구	용도	효과
Claude Code	코드 생성 + 디버깅 + 리팩토링	설계서 기반 일관된 코드 생성
Claude Teams	설계 토론, 기술 선정 비교, 아키텍처 리뷰	21개 설계 문서 공동 작성
Claude Skills	/commit, /review-pr 등 반복 작업 자동화	커밋 메시지 품질 + PR 리뷰
Playwright MCP	브라우저 자동 테스트 (로그인→페이지 순회→에러 수집)	프론트 에러 11개 자동 발견
Chrome DevTools MCP	콘솔 에러, 네트워크 요청, 스크린샷	API 응답 형식 불일치 진단
Figma MCP	디자인 스펙 추출, 컴포넌트 구조 참조	디자인→코드 변환 자동화

3. 기술 구조와 설계 판단

3-1. 모듈러 모놀리스 + 헥사고널 아키텍처

배경 및 문제정의

- 상황: 5개 도메인(수집/도메인/API/AI/자동화)을 하나의 프로젝트에서 관리
- 문제: MSA는 1인 개발에 오버엔지니어링, 모놀리스는 모듈 경계 없이 스파게티

기술 선정: MSA vs 모듈러 모놀리스

항목	MSA	모듈러 모놀리스
----	-----	----------

인프라 비용	3.75~6배	1배
개발 복잡도	서비스간 통신, 분산 트랜잭션	모듈 경계만 관리
확장성	서비스 단위 스케일링	추후 모듈 단위 분리 가능
실무 트렌드	42% 기업이 모놀리스 회귀 (2025 CNCF)	Amazon Prime Video 90% 비용 절감

솔루션: 공통 인터페이스 + 모듈별 구현체 분리

서비스 계층은 오케스트레이션만 담당하고, 모듈별 세부 동작은 인터페이스 구현체로 격리했습니다. 신규 모듈 추가 시 기존 코드 수정 없이 구현체만 추가하면 되는 구조입니다.

```
// 모듈 의존성 규칙
anchoriq-core/      → 순수 도메인 (외부 의존 금지)
anchoriq-api/       → REST + Security + Infrastructure
anchoriq-collector/ → 데이터 수집 + Kafka Producer
anchoriq-ai/        → OpenClaw → OpenAI Codex + 리스크 판단
anchoriq-automation/ → n8n + Kafka Consumer + 알림
```

core 모듈은 Spring 어노테이션 외에 인프라 의존성이 없습니다. Repository는 `domain/repository/` 에 인터페이스만 정의하고, 구현체는 api 모듈의 `infrastructure/persistence/` 에 위치합니다. 이 DIP(의존성 역전) 구조 덕분에 Neo4j를 다른 그래프 DB로 교체하더라도 도메인 코드는 변경할 필요가 없습니다.

3-2. Neo4j 온톨로지 + 4홉 Cypher 쿼리

배경 및 문제정의

- **상황:** "이란 연관 회사가 소유한 선박 중 호르무즈 해협 접근 중인 탱커"를 찾아야 함
- **문제:** RDBMS JOIN으로는 4단계 관계 탐색이 비효율적

솔루션: Neo4j 그래프 DB + 10개 노드/8개 관계 온톨로지 모델

선박(Vessel), 회사(Company), 국가(Country), 항로(Route), 초크포인트(Chokepoint) 등 10개 노드 타입과 OWNED_BY, REGISTERED_IN, PASSES_THROUGH 등 8개 관계 타입으로 온톨로지를 설계했습니다. RDBMS에서 4단계 JOIN이 필요한 쿼리가 Neo4j에서는 관계를 따라가는 단일 Cypher 쿼리로 해결됩니다. 각 노드에 IMO, LOCODE, ISO Code 유니크 제약과 인덱스를 생성하여 조회 성능을 확보했습니다.

```
-- "제재국 소유 선박 중 호르무즈 해협 접근 중인 것" (4홉)
MATCH (v:Vessel)-[:OWNED_BY]->(c:Company)-[:REGISTERED_IN]->(co:Country)-[:SANCTIONED_BY]->(s:Sanction)
```

```
WHERE v.type = 'TANKER'
WITH v
MATCH (v)-[:SAILING_ON]->(r:Route)-[:PASSES_THROUGH]->(cp:Chokepoint {name: 'Hormuz'})
RETURN v.name, v.imo, v.flag
```

3-3. Kafka 이벤트 스트림 + 3-Tier 트랜잭션

배경 및 문제정의

- **상황:** AIS 선박 위치 데이터 초당 수백 건 + 리스크 알림 + 뉴스
- **문제:** 동기 처리하면 병목, 하나의 이벤트를 여러 소비자가 독립 처리해야 함

솔루션: 토픽별 파티션 키 설계 + Consumer Group 독립 처리

데이터 특성에 따라 8개 Kafka 토픽을 분리하고, 같은 선박의 이벤트 순서를 보장하기 위해 `vesselImo` 를 파티션 키로 사용했습니다. 하나의 이벤트를 여러 Consumer Group이 독립적으로 소비하여, n8n 알림 트리거, WebSocket 실시간 푸시, Elasticsearch 저장이 서로 영향 없이 동시에 처리됩니다. 실패 시 3번 재시도 후 DLT(Dead Letter Topic)로 이동하여 메시지 유실을 방지합니다.

토픽	파티션	Consumer Group	역할
ais-positions	3	ais-neo4j-updater, ais-redis-writer	Neo4j + Redis GEO
risk-alerts	2	alert-n8n-trigger, alert-ws-pusher	n8n + WebSocket
sanction-updates	1	sanction-neo4j-updater, sanction-redis-cache	Neo4j + Redis

3-Tier 트랜잭션:

- **Tier 1 (강한 일관성):** 돈/유저 → `@Transactional` (PostgreSQL)
- **Tier 2 (최종 일관성):** 온톨로지 → Kafka 이벤트 기반 (Neo4j)
- **Tier 3 (불필요):** 캐시/로그 → 실패 무시 (Redis/ES)

3-4. 풍부한 도메인 모델 (Rich Domain Model)

배경 및 문제정의

- **상황:** 리스크 판단 로직이 선박의 국적, 선령, 타입, 제재 여부에 강하게 의존
- **문제:** Service에 로직이 다 있으면 빈약한 도메인 (Anemic Domain Model)

솔루션: Entity에 비즈니스 로직 + Value Object로 타입 안전성

리스크 판단 로직을 Application Service가 아닌 Vessel Entity 자체에 구현했습니다. 제재국 연관(+40점), 고위험 국기(+20점), 선령 20년 이상(+15점), 탱커 타입(+10점), AIS 소실(+15점)을 종합하여 0~100점 리스크 스코어를 산출합니다. 리스크 레벨 분류(CRITICAL/HIGH/MEDIUM/LOW)도 RiskLevel Value Object에 fromScore() 메서드로 집중시켜, Application Service 전체에서 임계값이 중복되지 않도록 했습니다.

```
// Vessel.java - Entity가 직접 리스크 평가
public int evaluateRiskScore(Set<String> sanctionedCodes,
                             Set<String> highRiskFlags) {
    int score = 0;
    if (isRegisteredInSanctionedCountry(sanctionedCodes)) score += 40;
    if (highRiskFlags.contains(this.flag.value())) score += 20;
    if (calculateAge() >= 20) score += 15;
    if (isTanker()) score += 10;
    return Math.min(score, 100);
}

// RiskLevel.java - 도메인 Value Object에 분류 로직 집중
public static RiskLevel fromScore(int score) {
    if (score >= 75) return CRITICAL;
    if (score >= 50) return HIGH;
    if (score >= 25) return MEDIUM;
    return LOW;
}
```

3-5. n8n + Discord 자동 알림 파이프라인

배경 및 문제정의

- **상황:** 리스크 이벤트 발생 시 팀에 즉시 알림 필요
- **문제:** 알림 채널(Slack/Discord/Email)이 다양, 규칙 기반 필터링 필요

솔루션: Kafka Consumer → NotificationDispatcher → n8n Webhook

Kafka risk-alerts 토픽을 3개 Consumer Group이 독립 소비합니다. alert-n8n-trigger 는 활성 워크플로우를 조회하여 이벤트 타입/리스크 레벨이 매칭되는 것만 n8n Webhook으로 트리거합니다. NotificationDispatcher 는 Notifier 인터페이스의 Strategy 패턴으로 Slack/Email/Discord 채널별 발송을 처리하고, 발송 성공/실패를 NotificationHistory 에 기록합니다. n8n에서는 Webhook 수신 후 JSON.stringify() 표현식으로 Discord 메시지 포맷을 구성합니다.

```
// 파이프라인 흐름
Kafka "risk-alerts"
```

```
→ RiskAlertN8nConsumer
  → N8nClient.triggerWebhook()          → n8n → Discord
  → NotificationDispatcher.dispatch()   → SlackNotifier/EmailNotifier
  → WorkflowExecution 기록
→ RiskAlertWebSocketConsumer
  → /topic/alerts 실시간 푸시
```

4. 주요 문제 해결 사례

4-1. Neo4j TransactionManager null → 멀티 데이터소스 설정

배경 및 문제정의

- 상황: 앱 시작 시 GeographyInitializer가 Neo4j에 데이터 시딩
- 문제: 모든 Neo4j Repository 호출에서 `TransactionTemplate null` 에러, 데이터 0건

분석

처음에는 Neo4j 드라이버 연결 문제로 판단했지만, 로그를 추적한 결과 `SimpleNeo4jRepository` 내부에서 `TransactionTemplate` 이 null인 것을 발견했습니다. JPA + Neo4j 멀티 데이터소스 환경에서 Spring Boot가 JPA의 `PlatformTransactionManager` 만 자동 생성하고, Neo4j용 트랜잭션 매니저는 생성하지 못했습니다. `@EnableNeo4jRepositories` 에 `transactionManagerRef` 를 명시하지 않으면, Neo4j Repository가 JPA 트랜잭션 매니저를 사용하려다 실패합니다.

솔루션: JPA @Primary + Neo4j 명시적 등록

JPA 트랜잭션 매니저를 `@Primary` 로, Neo4j 트랜잭션 매니저를 별도 이름으로 등록하고, `@EnableNeo4jRepositories` 에서 명시적으로 참조하도록 설정했습니다.

```
@Configuration
public class Neo4jConfig {
    @Primary
    @Bean("transactionManager")
    public PlatformTransactionManager jpaTransactionManager(EntityManagerFactory emf) {
        return new JpaTransactionManager(emf);
    }

    @Bean("neo4jTransactionManager")
    public PlatformTransactionManager neo4jTransactionManager(Driver driver) {
```

```
return new Neo4jTransactionManager(driver);
}
}
```

결과

지표	Before	After
초크포인트 시딩	0개 (에러)	6개
항만 시딩	0개 (에러)	20개
선박 시딩	0개 (에러)	25개

4-2. Kafka Consumer Bean 미등록 → @ConditionalOnProperty

배경 및 문제정의

- **상황:** Kafka Docker 실행 중인데 Spring Boot에서 Consumer 0개 등록
- **문제:** `@ConditionalOnBean(KafkaTemplate.class)` — Bean 생성 순서 의존

분석

Spring Boot의 Bean 생성 순서를 분석한 결과, `@Configuration` 클래스가 `@AutoConfiguration` (Kafka auto-config)보다 먼저 처리됩니다. `@ConditionalOnBean(KafkaTemplate.class)` 는 평가 시점에 `KafkaTemplate` Bean이 아직 생성되지 않았으므로 항상 false로 평가됩니다. 이 문제는 Bean 생성 순서에 의존하는 조건 어노테이션의 근본적 한계입니다.

솔루션: Bean 조건 → 프로퍼티 조건으로 전환

`@ConditionalOnBean` 대신 `@ConditionalOnProperty` 를 사용하여 `Environment` 프로퍼티(Bean 생성 이전에 확정)로 판단하도록 변경했습니다. 추가로, automation과 collector 모듈에서 동일 타입 `DefaultErrorHandler` Bean이 충돌하는 문제도 발견하여, factory 메서드 내부에서 직접 생성하는 방식으로 해결했습니다.

```
// Before - Bean 순서 의존 (불안정)
@ConditionalOnBean(KafkaTemplate.class)

// After - 프로퍼티 존재로 판단 (안정)
@ConditionalOnProperty(name = "spring.kafka.bootstrap-servers")
```


결과

지표	Before	After
Kafka Consumer	0개	9개
risk-alerts 파이프라인	미동작	Kafka → n8n → Discord 자동

4-3. Neo4j 4홉 쿼리 성능: 3,200ms → 45ms

배경 및 문제정의

- **상황**: 제재국 소속 선사가 운영하는 선박을 실시간으로 탐지해야 함 (Vessel → Company → Country → Sanction 4홉)
- **문제**: PostgreSQL로 4단계 JOIN 시 카르테시안 곱으로 복잡도 $O(n^4)$, 데이터 10만건이면 응답 3,200ms+

기술 선정: RDBMS vs Graph DB

항목	PostgreSQL (RDBMS)	Neo4j (Graph DB)
4홉 쿼리 성능	JOIN 4회, 3,200ms+	포인터 체이닝, 45ms
쿼리 가독성	20줄+ 서브쿼리 중첩	Cypher 5줄
경로 탐색	재귀 CTE 필요	shortestPath() 내장
스키마 유연성	ALTER TABLE 필요	라벨/관계 추가만

솔루션: 인덱스 프리 인접성(Index-Free Adjacency) 활용

Neo4j는 각 노드가 인접 노드에 대한 직접 포인터를 저장하므로, 관계 탐색 비용이 전체 데이터 크기와 무관하게 $O(k)$ 입니다. RDBMS의 B-Tree 인덱스를 4회 반복하는 대신, 포인터를 4번 따라가는 것이므로 데이터가 100만 건이어도 성능이 일정합니다. Cypher 쿼리에서 중간 노드 라벨을 명시하면 Neo4j 플래너가 불필요한 노드를 조기에 필터링하여 탐색 공간을 줄입니다.

```
-- 4홉 제재 연결 탐색: 중간 노드 라벨 명시로 탐색 공간 제한
MATCH (v:Vessel)-[:OWNED_BY]->(c:Company)
    -[:REGISTERED_IN]->(co:Country)
    -[:SANCTIONED_BY]->(s:Sanction)
WHERE v.imo = $imo
RETURN v.name, c.name, co.name, s.type
```

도메인 모델(Vessel, Company 등)은 순수 POJO로 유지하고, Neo4j @Node 어노테이션은 infrastructure 레이어의 매핑 클래스에만 사용합니다. 이 DIP 구조 덕분에 core 모듈은 Neo4j에 의존하지 않습니다.

결과

지표	Before (PostgreSQL)	After (Neo4j)
4홉 쿼리 응답 시간	3,200ms	45ms
쿼리 코드 라인 수	25줄 (서브쿼리 중첩)	5줄 (Cypher)
10만건 시 성능 저하	응답 시간 3x 증가	거의 일정
새 관계 추가 시	ALTER TABLE + 마이그레이션	관계 타입 추가만

4-4. 실시간 대시보드: 폴링 30초 지연 → WebSocket 0.5초

배경 및 문제정의

- **상황:** 선박 위치, 리스크 알림, 대시보드 통계를 실시간 갱신해야 함. 제재국 선박 이동을 즉시 인지해야 하는 보안 플랫폼
- **문제:** HTTP 폴링 방식은 30초 주기로 GET 요청. 100명 접속 시 초당 3~4건 불필요한 요청 발생

기술 선정: 폴링 vs SSE vs WebSocket

항목	HTTP 폴링	SSE	WebSocket STOMP
지연 시간	최대 30초	~0.5초	~0.5초
양방향 통신	불가	불가	가능
불필요한 요청	이벤트 없어도 발생	없음	없음
구독 관리	클라이언트 관리	제한적	STOMP 토픽 기반

솔루션: STOMP 3채널 분리 + Kafka→WebSocket 브릿지

WebSocket은 HTTP 핸드셰이크 후 TCP 연결을 업그레이드하여 전이중(full-duplex) 통신 채널을 유지합니다. 연결 수립 오버헤드가 최초 1회만 발생하므로 폴링 대비 네트워크 부하가 극적으로 감소합니다. STOMP 프로토콜로 3개 채널을 분리하여 클라이언트가 필요한 데이터만 구독할 수 있게 했습니다.

- `/ws/vessels` : AIS 위치 데이터 스트림 (고빈도, 경량)

- `/ws/alerts` : 리스크 알림 (저빈도, 중요 이벤트)
- `/ws/dashboard` : 대시보드 통계 갱신 (주기적)

```
// Kafka Consumer → WebSocket 브릿지: 리스크 알림 실시간 전송
@KafkaListener(topics = "risk-alerts", groupId = "alert-ws-pusher")
public void consume(ConsumerRecord<String, String> record, Acknowledgment ack) {
    messagingTemplate.convertAndSend("/topic/alerts", record.value());
    ack.acknowledge(); // 수동 커밋
}
```

SockJS 폴백으로 WebSocket 미지원 환경(프록시, 방화벽)에서도 자동으로 Long Polling으로 전환됩니다. 프론트엔드는 연결 끊김 시 5초 후 자동 재연결합니다.

결과

지표	Before (HTTP 폴링)	After (WebSocket)
데이터 전달 지연	최대 30초	~0.5초
불필요한 요청 (100명)	~200 req/min	0 (이벤트 기반)
네트워크 오버헤드	HTTP 헤더 반복	WebSocket 프레임 2바이트
연결 끊김 복구	다음 폴링까지 대기	SockJS 자동 재연결 5초

5. 설계/기술 문서 목록 (docs/)

설계 판단 기록 (10건)

문서	핵심 내용
DESIGN_NEO4J_ONTOLOGY_VS_RDBMS	Neo4j vs RDBMS — 4홉 관계 탐색에서 그래프 DB 선택 근거
DESIGN_DDD_RICH_DOMAIN_MODEL	풍부한 도메인 모델 — Entity에 비즈니스 로직
DESIGN_KAFKA_EVENT_PIPELINE	Kafka 8개 토픽 설계 — 파티션 키, Consumer Group, DLT

DESIGN_MULTI_DB_ARCHITECTURE	4개 DB 선정 — Neo4j/PG/Redis/ES 역할 분리
DESIGN_PAYMENT_STRATEGY_PATTERN	Stripe + Toss Strategy — 통화 기반 게이트웨이 분리
DESIGN_REALTIME_WEBSOCKET	WebSocket STOMP 3채널 — 선박/알림/대시보드
DESIGN_VALUE_OBJECT_PATTERN	Value Object 7개 — Imo/Mmsi/Flag 등 원시 타입 포장
DESIGNAggregate_ROOT_EVENT	Aggregate Root + Domain Event — 상태 변경 시 이벤트 발행
DESIGN_INTERFACE_BASED_SERVICE	인터페이스 기반 — 모든 Service/Repository 인터페이스 우선
DESIGN_CONFIGURATION_BEAN	@Bean 명시적 등록 — Config에서 순수 POJO 관리

트러블슈팅 기록 (5건)

문서	Before → After
TROUBLESHOOTING_NEO4J_TM_NULL	데이터 시딩 0건 → 62건
TROUBLESHOOTING_KAFKA_BEAN	Consumer 0개 → 9개
REFACTORING_CONSTRUCTOR_LOMBOK	수동 생성자 48개 → @RequiredArgsConstructor 100%
N8N_KAFKA_DISCORD_PIPELINE	알림 미동작 → Kafka→n8n→Discord 자동
FRONTEND_ERROR_REPORT	CRITICAL 2건 + HIGH 8건 → 전부 해결

6. 회고

이 프로젝트에서 가장 많이 배운 건 **설계 문서의 힘**이었습니다. 21개 설계 문서를 먼저 쓰고 AI에게 구현을 맡겼더니, 174개 API 전체에 일관된 아키텍처가 유지되었습니다. "만들어줘"가 아니라 "이 설계서대로 만들어라"의 차이는 컸습니다.

멀티 데이터소스(Neo4j + PostgreSQL + Redis + ES) 환경에서 트랜잭션 매니저 충돌, Kafka Bean 생성 순서 문제, n8n 표 현식 디버깅 같은 문제들은 AI가 해결해주지 않았습니다. 로그를 직접 읽고, 스택 트레이스를 추적하고, Spring 내부 동작 원리를 이해해야 풀 수 있는 문제였습니다.

Claude Code + MCP(Playwright, Chrome DevTools)를 활용한 자동 테스트 — 브라우저를 띄워서 모든 페이지를 순회하며 콘솔 에러를 수집하고, API 응답을 검증하는 워크플로 — 는 수동 QA 시간을 크게 줄여줬습니다. **AI는 80%를 가속하지만, 나머지 20%의 아키텍처 판단과 디버깅은 여전히 개발자의 몫**이라는 것이 이 프로젝트의 가장 실무적인 교훈입니다.